

뜨개더 iOS 앱 QA

유하은 / iOS 앱 기능과 데이터 정합성 검증

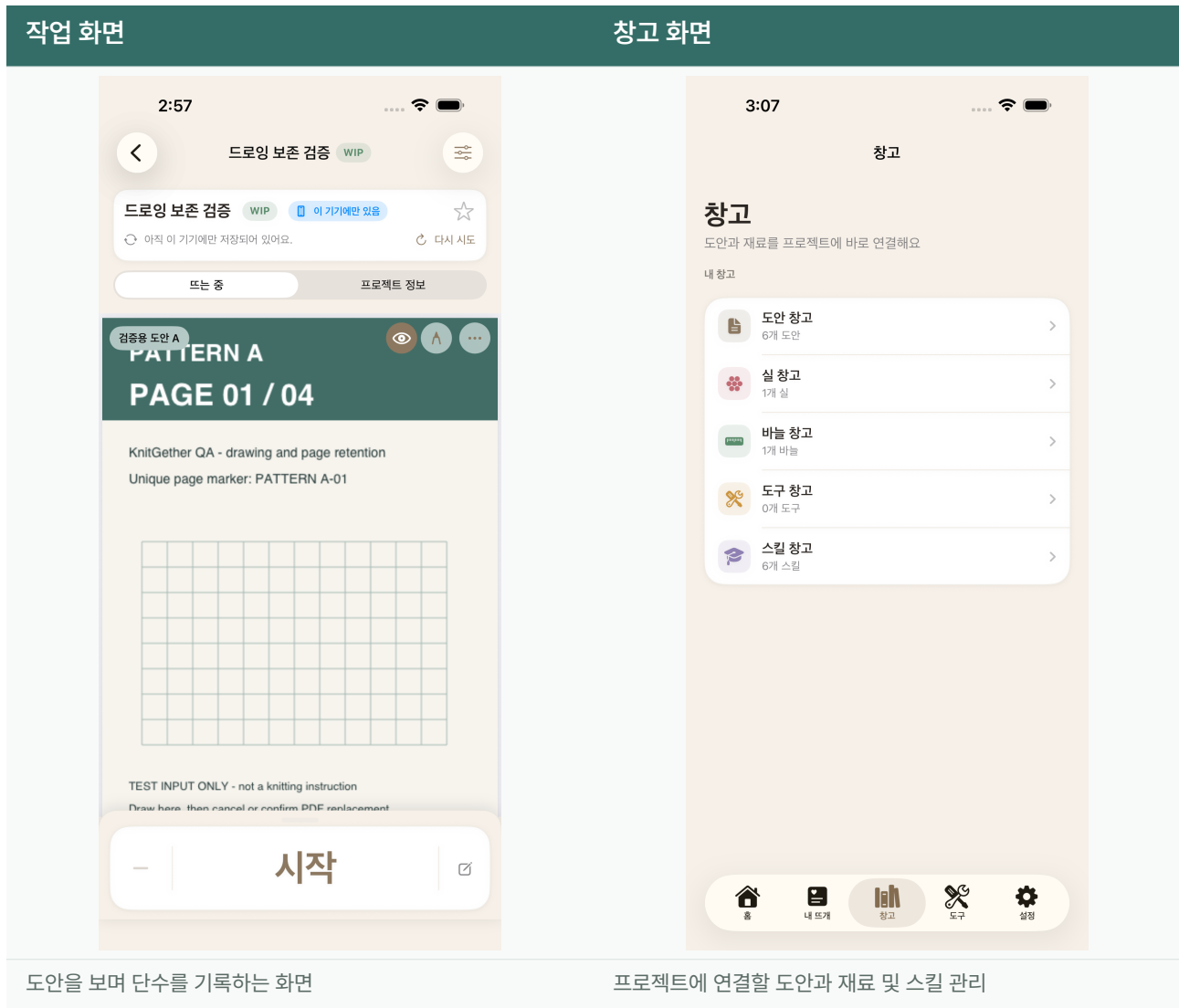
문서 구성

- 01 프로젝트와 역할
- 02 테스트 전략과 설계
- 03 주요 검증 사례 5개
- 04 결과와 회고

01 프로젝트와 역할

기획 배경

뜨개질 중 도안과 기술 설명을 오가면 작업 위치를 다시 찾아야 하고, 실과 바늘, 도구 정보도 따로 관리하게 됩니다. 이를 한 프로젝트 안에서 관리하도록 뜨개더를 기획했습니다. 도안 보기, 단수 카운터, 작업 기록을 중심으로 도안과 실, 바늘, 도구, 스킬 창고를 연결합니다.



테스트 데이터를 준비해 iOS 시뮬레이터에서 촬영한 앱 화면입니다.

01 프로젝트와 역할

담당 역할

앱 기획

- 뜨개 작업에 필요한 기능 정의
- 프로젝트와 참고 및 작업 화면의 연결 관계 설계

테스트 설계

- 기능별 테스트 시나리오 설계
- AI의 구현 분석을 참고해 실제 사용 흐름에 맞는 기대 결과와 검증 조건 정의
- Appium 자동화 대상 기능과 테스트 조건 정의

QA 수행

- 앱 수동 테스트 수행
- Postman API 응답 확인과 SQL 저장값 대조
- Charles 응답 중단과 요청 변경 상황에서 앱의 기록 유지 여부와 오류 안내 확인
- Appium 테스트 코드 보완, 실행 및 실패 결과 확인

앱과 API 서버 구현에는 AI 도구를 활용했습니다.

주요 검증 관점

- 작업 기록 보존: 앱 재실행, 서버 응답 중단, 도안 교체 취소 시 단수와 작업 기록 및 드로잉이 유지되는지 확인
- 데이터 정합성: API 응답과 DB 저장값을 대조하고, 프로젝트와 단수 정보의 연결 관계 확인

검증 대상과 환경

- 검증 대상: SwiftUI iOS 앱, 앱 로컬 저장소, NestJS API 서버, PostgreSQL DB
- 수동 테스트: iPhone 실기기와 iOS 시뮬레이터
- Appium UI 자동화: iOS 시뮬레이터

[GitHub: 앱 코드와 QA 상세 기록](#)

02 테스트 전략과 설계

검증 방법과 수정 후 확인

사용 흐름과 예외 상황을 기준으로 테스트 케이스를 설계했습니다. 이후 기존 검증 항목과 추가 기능을 Appium UI 테스트 및 API 테스트로 확장했습니다. 수동 검증이 필요한 항목은 유지하고, 수정 후에는 관련 테스트를 다시 실행했습니다.

프로젝트에서 사용한 검증 방법과 도구는 다음과 같습니다.

검증 방법	도구와 환경	확인 내용
앱 수동 테스트	iPhone과 iOS 시뮬레이터	사용 흐름, 화면 이동, 계정 변경 후 상태
API 수동 확인	Postman	요청을 보내고 응답 코드와 반환 데이터 확인
통신 조건 제어	Charles	요청과 응답 확인, 응답 중단, 요청값 변경 후 앱 동작 확인
DB 직접 확인	PostgreSQL	SQL로 저장값, 소유자와 연결 관계 확인 및 오류 조건 생성
UI 자동화	Appium	화면 조작 후 표시와 상태 유지 확인
API 자동화	pytest	API 요청과 SQL 검증을 자동 실행해 응답과 저장값 확인
앱 내부 기능 자동화	Swift Testing	계정별 저장 폴더 삭제 등 내부 기능을 직접 실행해 결과 확인

02 테스트 전략과 설계

테스트 범위와 케이스 구성

수동 검증과 이후 추가한 UI 및 API 테스트의 설계 범위를 다음과 같이 정리했습니다.

영역	설계한 테스트 내용
가입과 로그인	온보딩, 입력 조건, 로그인 실패, 로그아웃, 계정 구분
프로젝트 관리	생성, 수정, 삭제, 정렬, 필터, 즐겨찾기
작업 화면	도안, 드로잉, 단수, 스킬 조회와 화면 복귀 후 상태 유지
참고	실과 바늘 및 도구의 등록, 수정, 삭제와 프로젝트 연결
저장과 동기화	통신 중단, 연결 복구, 여러 기기의 수정 충돌, 앱 재실행, 저장 거부 시 기록 보존

이 중 사용 흐름과 예외 상황을 다룬 22건의 설계 묶음은 주요 사용 흐름과 기능 간 연결 4건, 기록 유실과 계정 분리의 예외 상황 6건, 기능별 동작과 예외 처리 12건으로 구성했습니다. 이후 추가한 UI 및 API 자동화 테스트는 별도로 관리했습니다.

위험도에 따른 테스트 우선순위

아래 테스트 설계에서는 다음 두 문제를 높은 우선순위(P1)로 두고, 별도의 예외 상황 테스트를 설계했습니다.

1. 작업 기록 유실: 통신이 끊겼다가 복구되거나 여러 기기에서 수정할 때, 사용자가 쌓아 온 기록이 사라질 수 있어 중점 검증 대상으로 선정
2. 계정 간 데이터 노출: 계정을 바꾼 뒤 이전 계정의 기록이 보이거나 다른 계정으로 업로드될 수 있어 중점 검증 대상으로 선정

02 테스트 전략과 설계

대표 테스트 조건과 기대 결과

검증 관점	테스트 조건	기대 결과
단수 기록 보존	단수 변경 후 저장 성공 시점과 요청 처리 중 시점에 각각 앱 강제 종료 → 재실행	같은 프로젝트에서 종료 직전 변경한 단수 유지
통신 중단 중 생성한 프로젝트 보존	API 서버 연결 중단 → 프로젝트 생성 → 연결 복구 → 목록 조회	앱에 저장한 프로젝트가 유지되고, 서버에도 같은 ID와 입력값으로 1건 저장
계정 변경 후 기록 분리	A 계정에서 프로젝트 확인 → 로그아웃 → B 계정 로그인	B 계정의 화면에 A 계정의 프로젝트가 표시되지 않음
다른 계정으로의 업로드 방지	A 계정에서 통신 중단 중 프로젝트 생성 → 연결 복구 후 A 로그아웃 → B 로그인 → 목록 조회	A 계정에서 만든 미업로드 프로젝트가 B 계정 소유로 서버에 저장되지 않음
도안 교체 취소 시 기록 보존	기존 도안의 드로잉이 저장되어 있는지 확인 → 도안 교체 시도 → 삭제 안내에서 취소	기존 도안과 드로잉 유지
도안 교체 승인 시 드로잉 삭제	기존 도안의 드로잉이 저장되어 있는지 확인 → 삭제 안내를 확인하고 교체 승인	새 도안으로 교체되고 기존 드로잉이 제거되며, 앱 재실행 후에도 다시 나타나지 않음

근거: 품질 리스크맵 / 근거: 테스트 조건과 기대 결과 / 근거: UI와 API 테스트 범위 및 기존 케이스 연결

03 주요 검증 사례

03-1 도안 교체 시 드로잉 유실 (DEF-10)

배경과 검증 목적

PDF 도안 위에 드로잉이 있는 상태에서 다른 도안으로 교체하면, 삭제 안내나 확인창 없이 기존 드로잉이 사라졌습니다. 사용자가 기록이 삭제된다는 사실을 확인하고 교체를 취소할 기회가 없는 문제였습니다.

테스트 조건과 기대 결과

드로잉이 저장된 PDF 도안을 준비하고, 다른 PDF로 교체하는 상황을 테스트 조건으로 정했습니다. 교체 전에 드로잉이 실제로 존재하는지 확인하고, 삭제 안내와 사용자의 선택에 따라 기대 결과를 나눴습니다.

테스트 조건	기대 결과
교체 전	기존 도안에 저장한 드로잉 존재
교체 시도	드로잉 삭제 안내와 확인창 표시
교체 취소	기존 도안과 드로잉 유지
삭제 안내 후 승인	새 도안으로 교체되고 이전 드로잉 제거

실제 결과와 QA 판단

수정 전에는 확인창 없이 도안이 즉시 교체되고 기존 드로잉이 삭제됐습니다. 도안 파일 교체가 성공하더라도 사용자가 남긴 기록이 예고 없이 없어지므로, 화면 표시 문제와 구분해 데이터 유실로 판단했습니다. 기존 결함 기록에서도 데이터 유실로 분류를 변경했습니다.

03-1 도안 교체 시 드로잉 유실 (DEF-10)

후속 조치와 재검증

도안을 교체하기 전에 기존 드로잉이 삭제된다는 안내를 보여주고, 사용자가 취소하거나 교체를 진행할 수 있도록 확인창이 추가됐습니다.

Appium 자동화 테스트에는 다음 확인 절차를 반영했습니다.

1. 도안에 드로잉을 남기고 실제로 존재하는지 확인
2. 도안 교체를 시도한 뒤 취소하면 드로잉이 유지되는지 확인
3. 다시 교체를 시도하고 승인하면 기존 드로잉이 제거되는지 확인

교체 취소와 승인에 따른 드로잉 상태를 자동으로 확인하고, 이후 앱을 수정할 때도 같은 조건으로 반복 검증할 수 있도록 구성했습니다.

이 사례에서 배운 점

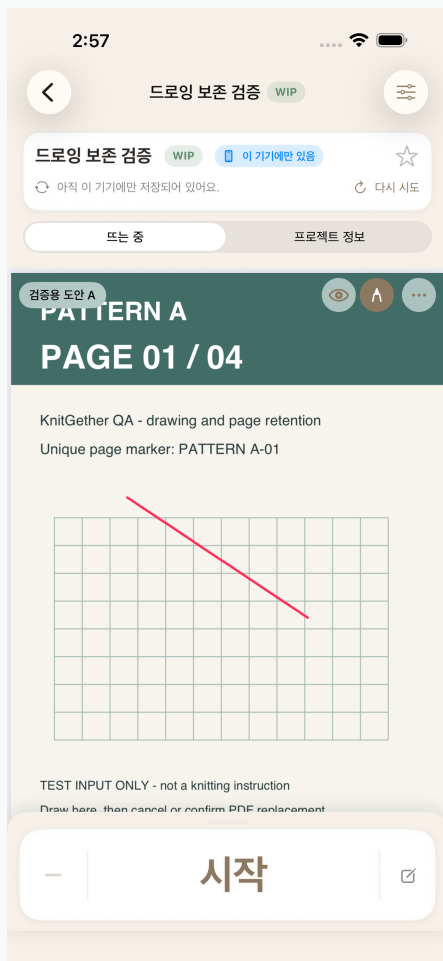
파일 교체의 성공 여부와 함께, 기존 기록이 어떤 선택을 거쳐 삭제되는지 확인해야 합니다. 취소 경로도 데이터 보존을 검증하는 핵심 테스트입니다.

근거: DEF-10 재현, 분류, 수정 기록 / 회귀 코드: UI-45 도안 교체 취소와 승인

03-1 추가 재검증 화면

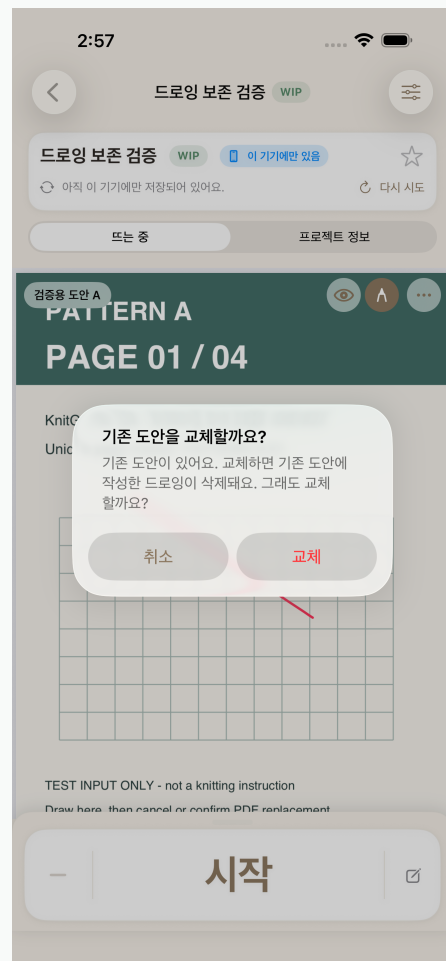
추가 검증에서는 테스트용 도안 A와 B로 PDF 직접 교체 경로를 다시 실행했습니다. 삭제 안내, 취소 후 보존, 승인 후 제거를 확인했고, 앱을 재실행한 뒤에도 도안 B와 드로잉 제거 상태가 유지됐습니다.

① 교체 전



도안 A 위에 드로잉을 남긴 상태

② 교체 시도

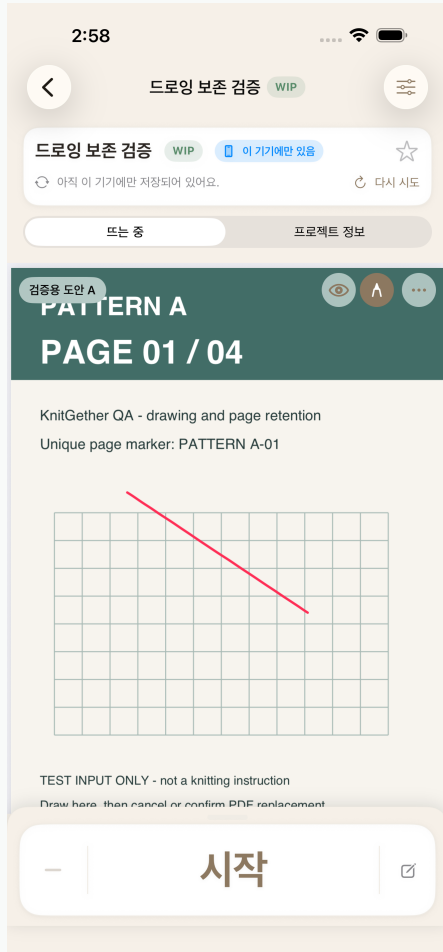


교체 전에 드로잉 삭제 안내 표시

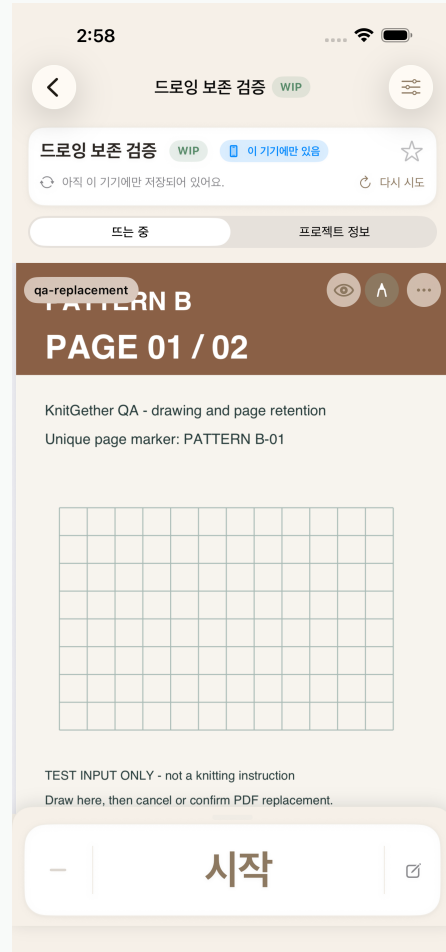
03-1 추가 재검증 화면

③ 취소 후

④ 승인 후



기존 도안 A와 빨간 드로잉 유지



갈색 도안 B로 변경되고 드로잉 제거

촬영: 2026-10-04, iOS 26.5 시뮬레이터, Appium. [재실행 화면과 실행 기록](#)

03-2 단수 정보 누락으로 프로젝트 목록 조회 실패 (DEF-08)

배경과 검증 목적

단수 정보가 누락된 프로젝트 한 건 때문에 목록 전체의 조회가 실패했습니다. 불완전한 데이터가 정상 프로젝트 조회에 미치는 영향을 확인했습니다.

테스트 조건과 기대 결과

정상 프로젝트와 오류 대상 프로젝트를 준비하고, DB에서 오류 대상의 단수 정보만 제거했습니다. 정상 프로젝트는 계속 조회할 수 있어야 한다는 기준으로 API 응답을 비교했습니다.

테스트 조건	기대 결과
단수 정보가 누락된 프로젝트와 정상 프로젝트를 함께 조회	한 항목의 오류가 목록 전체의 조회 실패로 이어지지 않음
목록 응답의 프로젝트 ID 확인	정상 프로젝트 포함, 오류가 있는 프로젝트 제외
API 응답과 앱에 표시된 값 비교	이전에 저장한 목록 표시와 최신 조회 성공을 구분할 수 있음

실제 결과와 QA 판단

수정 전 서버는 단수 정보가 없는 프로젝트를 응답으로 변환하는 과정에서 오류를 내고, 목록 전체에 HTTP 500을 반환했습니다. 한 항목의 불완전한 데이터가 정상 항목의 조회까지 막는 문제였으며, 프로젝트 내부 기준에서 Critical로 분류했습니다.

앱에 이전 목록이 남아 있을 때는 화면만 보고 정상 조회로 오인할 수 있었습니다. 기존 실행 기록에서는 서버 단수를 99로 바꿔도 앱에 22가 표시돼, 화면에 남아 있는 값과 최신 서버 데이터가 다르다는 점을 확인했습니다. 이 때문에 화면 표시와 함께 API 응답 및 DB 값을 대조했습니다.

03-2 단수 정보 누락으로 프로젝트 목록 조회 실패 (DEF-08)

후속 조치와 재검증

응답을 만들 수 없는 항목만 목록에서 제외하도록 수정되었습니다. 동일한 DB 조건에서 수정 전후 서버의 응답을 비교했습니다. 수정 전에는 HTTP 500으로 목록 조회가 실패했고, 수정 후에는 HTTP 200과 정상 프로젝트가 반환되었습니다.

pytest와 SQL을 사용하는 API 자동화 테스트에는 다음 확인 절차를 반영했습니다.

1. 정상 프로젝트와 오류를 만들 프로젝트를 각각 생성
2. DB에서 오류 대상의 단수 정보를 삭제 상태로 변경
3. 목록 응답이 HTTP 200인지, 정상 프로젝트가 포함되고 오류 대상은 제외되는지 확인

최초 재현은 레코드 제거, 자동화는 삭제 상태 변경 조건입니다. 정상 목록 반환을 확인했으며, 제외된 항목의 복구와 안내 및 개별 조회 오류는 추가 확인 범위로 남겼습니다.

이 사례에서 배운 점

DB의 불일치를 발견하는 데서 그치지 않고, 그 상태가 API 응답과 사용자의 작업 진입에 어떤 영향을 주는지 연결해 판단했습니다.

근거: DEF-08 재현과 수정 전후 비교 기록 / 회귀 코드: API-21 정상 목록 보존

03-3 다른 탭에서 복귀하면 도안 페이지 초기화 (DEF-19)

배경과 검증 목적

여러 페이지로 된 도안을 보다가 홈 탭에 다녀오면, 작업 화면에서 보던 도안이 1페이지로 돌아갔습니다. 앱을 종료하지 않았는데도 사용자가 작업 위치를 다시 찾아야 하는 문제였습니다.

테스트 조건과 기대 결과

8페이지 도안의 3페이지를 보는 상태에서 이동 경로를 나눴습니다. 작업 화면에서 목록으로 나온 뒤 홈 탭을 거쳐 같은 프로젝트로 돌아오는 경로와, 사전이나 스킬을 조회한 뒤 복귀하는 경로를 구분했습니다.

테스트 조건	기대 결과
사전이나 스킬 조회 후 작업 화면 복귀	보던 도안 페이지 유지
목록과 홈 탭을 거쳐 같은 프로젝트 재진입	보던 도안 페이지 유지

앱 종료 후 재실행까지 페이지를 유지할지는 당시 요구사항의 보장 범위에서 제외했습니다.

실제 결과와 QA 판단

사전이나 스킬 조회 후 복귀할 때는 페이지가 유지됐지만, 홈 탭을 거쳐 돌아오면 3페이지에서 1페이지로 초기화됐습니다.

기존 요구사항에는 사전과 스킬을 조회한 뒤 돌아왔을 때 보던 페이지를 유지한다는 기준만 있었고, 홈 탭에 다녀오는 경우는 명시돼 있지 않았습니다. 사용자가 같은 프로젝트의 작업을 이어가는 상황이므로, 홈 탭을 거쳐 돌아와도 보던 페이지를 유지하도록 기대 결과를 추가했습니다. 이 기준과 달리 1페이지로 초기화되는 동작을 결함으로 기록했습니다.

구현 분석에서는 페이지 위치가 PDF 화면 내부에만 남고, 탭 이동 후 화면이 다시 만들어질 때 복원되지 않는 구조가 확인됐습니다. 같은 '화면 복귀'라도 이동 경로에 따라 결과가 달라지므로, 정상인 사전 조회 경로와 문제가 있는 탭 이동 경로를 별도 테스트로 유지했습니다.

03-3 다른 탭에서 복귀하면 도안 페이지 초기화 (DEF-19)

후속 조치와 재검증

보던 도안 페이지를 저장하고, 화면이 다시 열릴 때 그 페이지로 돌아오도록 수정되었습니다. 문제가 발생한 경로와 기존에 정상이던 경로를 나눠 Appium 테스트에 반영했습니다.

1. 문제가 발생한 경로: 도안을 넘긴 뒤 목록과 홈 탭을 거쳐 같은 프로젝트로 돌아오기
2. 기존에 정상이던 경로: 도안을 넘긴 뒤 사전을 조회하고 작업 화면으로 돌아오기

각 경로에서 이동 전후에 화면에 보이는 도안 본문 텍스트가 같은지 비교했습니다. 수정 후 재검증 기록에서는 홈 탭 이동 후 복귀와 사전 조회 후 복귀 테스트가 모두 통과했습니다. 수정한 경로뿐 아니라 기존 경로도 함께 반복 확인하도록 구성했습니다.

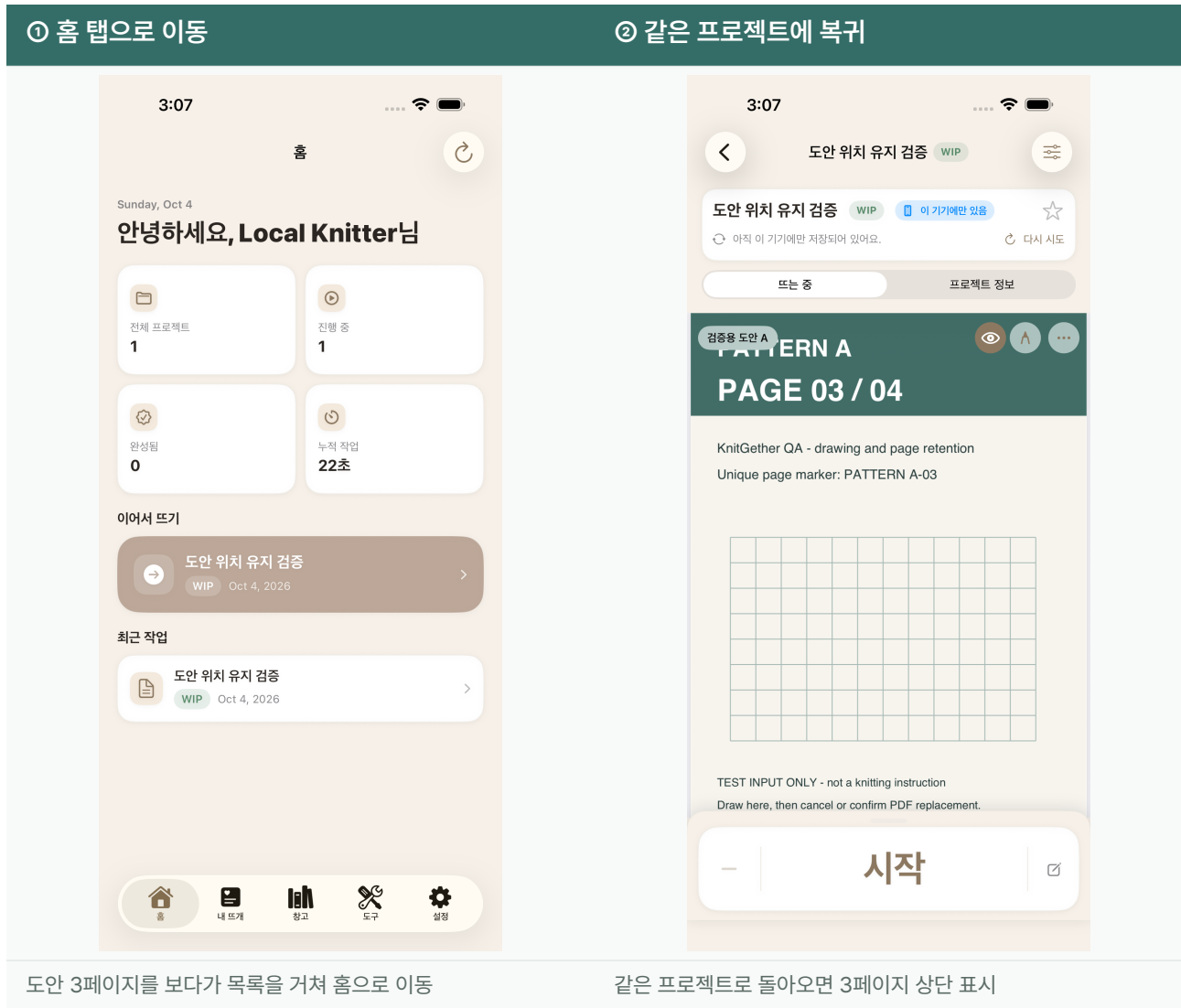
이 사례에서 배운 점

‘돌아왔을 때 유지된다’는 조건에는 어디로 이동했다가 돌아오는지도 포함해야 합니다. 화면 이동과 앱 재실행은 조건이 다르므로 검증 범위를 구분했습니다.

근거: DEF-19 발견과 기대 결과 확정 / 회귀 코드: UI-51 이동 경로별 도안 위치 유지

03-3 추가 재검증 화면

추가 검증에서는 페이지마다 고유 표시가 있는 4페이지 테스트 도안을 사용했습니다. 3페이지에서 목록과 홈 탭을 거쳐 같은 프로젝트로 돌아왔을 때, 같은 3페이지가 표시되는지 확인했습니다. 사전 조회 화면을 닫고 돌아온 뒤에도 3페이지가 유지됐습니다.



도안 3페이지를 보다가 목록을 거쳐 홈으로 이동

같은 프로젝트로 돌아오면 3페이지 상단 표시

촬영: 2026-10-04, iOS 26.5 시뮬레이터, Appium. 판정 기준은 동일한 페이지로의 복귀입니다. [이동 과정과 사전 복귀 화면 및 실행 기록](#)

03-4 계정 변경 후 이전 계정의 프로젝트 노출 (DEF-05~07)

배경과 검증 목적

A 계정에서 로그아웃한 뒤 B 계정으로 로그인했는데, 앱에 다른 계정의 프로젝트가 함께 표시됐습니다.

테스트 조건과 기대 결과

서로 구별되는 A와 B 계정의 프로젝트를 준비했습니다. 계정 변경 전후의 화면과 API 응답을 비교하고, 다른 계정으로 업로드되는지도 별도로 확인했습니다.

테스트 조건	기대 결과
A 로그아웃 후 B 로그인	B 계정의 프로젝트만 표시
B 계정의 API 응답과 앱 목록 비교	API와 화면 모두 B 계정의 소유자 범위 유지
A가 통신 중단 중 만든 프로젝트를 남긴 채 B로 변경	A의 프로젝트가 B 소유로 업로드되지 않음

실제 결과와 QA 판단

기존 실행 기록에서는 서버가 현재 계정의 프로젝트 4건을 반환했지만 앱에는 5건이 표시됐습니다. 서버 응답에 없는 프로젝트가 앱에 남아 있어, 휴대폰에 보관된 데이터를 확인했습니다.

앱은 통신이 끊겨도 기록을 볼 수 있도록 프로젝트를 휴대폰에도 저장합니다. 당시에는 로그아웃 시 이전 계정의 데이터가 미로그인 저장 영역으로 복사됐고, 다른 계정으로 로그인하면 이 데이터까지 표시됐습니다. 원인은 서버가 아닌 앱 내부의 계정 구분과 로그아웃 처리에 있었습니다.

다른 계정의 기록이 화면에 노출된 것은 결함으로 판단했습니다. 다만 다른 계정으로의 업로드 요청은 0건이었고, 서버에도 새 프로젝트가 생성되지 않았습니니다. 관련된 DEF-05~07은 하나의 사례로 묶었습니다.

03-4 계정 변경 후 이전 계정의 프로젝트 노출 (DEF-05~07)

후속 조치와 재검증

로그아웃한 계정의 저장 데이터를 정리하고, 미로그인 상태에서 이전 기록을 숨기도록 수정했습니다. 미업로드 기록은 먼저 업로드를 시도하며, 남은 기록이 있으면 안내 후 로그아웃 여부를 선택하도록 했습니다. 수정 결과는 화면과 저장소로 나눠 확인했습니다.

1. **Appium 화면 테스트:** 로그인 중 다른 계정의 프로젝트가 보이지 않는지, 로그아웃 후 계정 이름과 프로젝트가 숨겨지는지, 앱 재실행 후 로그인 상태가 되살아나지 않는지 확인
2. **Swift Testing을 이용한 내부 기능 테스트:** 임시 A 계정과 B 계정 폴더를 만들고 A의 데이터 삭제 함수를 직접 실행해, A 폴더만 삭제되고 B 폴더는 유지되는지 확인

자동화 범위는 위의 화면 상태와 저장 폴더 정리이며, A에서 B로 전환하는 모든 경로를 포함하지는 않습니다.

이 사례에서 배운 점

서버의 소유자 구분이 정상이어도 앱에 남은 데이터 때문에 다른 계정의 정보가 보일 수 있습니다. API 응답, 앱 화면, 앱 내부 저장 데이터를 함께 확인해야 합니다.

근거: 계정 변경 시 노출과 업로드 구분 / 회귀 코드: UI-77 계정 데이터 표시 확인 / 저장소 확인 코드: 계정별 저장 폴더 정리

03-5 앱 강제 종료 후 단수 기록 복구 (TC-CNT02-01)

배경과 검증 목적

단수를 변경한 뒤 앱이 종료되더라도 마지막 입력이 유지되는지 확인했습니다. 저장 성공 응답을 받은 뒤 종료한 경우와, 응답을 받기 전에 종료한 경우를 구분한 정상 동작 검증 사례입니다.

테스트 조건과 기대 결과

화면과 서버의 단수가 모두 0인 프로젝트에서 시작했습니다. 단수를 3까지 변경하고, Charles로 카운터 저장 요청과 응답을 확인하면서 종료 시점을 나눴습니다. 화면 값은 앱에서, 통신 상태는 Charles에서, 서버 저장값은 SQL로 확인하는 조건을 설계했습니다. 앱 내부 저장값도 확인 대상으로 포함했습니다.

테스트 조건	기대 결과
단수 3으로 변경 → 성공 응답 확인 → 앱 강제 종료 → 같은 프로젝트 재진입	화면과 서버에 단수 3 유지
세 번째 단수 저장 요청의 응답을 Charles에서 보류 → 앱 강제 종료 → 보류한 통신 폐기 → 재실행	화면과 앱 내부 저장값에 단수 3 유지
프로젝트 재조회 또는 저장 재개 후 서버 값 조회	서버도 최종적으로 3이 되고 앱의 값이 이전 값으로 되돌아가지 않음

실제 결과와 QA 판단

단수를 한 번 올릴 때마다 저장 API가 호출됐습니다. 앱 강제 종료 후 복구 검증의 최종 확인값은 화면, 앱 내부 저장소, 서버 DB에서 모두 3으로 일치했습니다. 마지막 입력이 유지된 결과를 근거로 PASS로 판정했습니다.

응답을 중단했다고 해서 서버 저장까지 취소됐다고 판단하지 않았습니다. 서버에 이미 3이 저장됐을 수도 있고, 아직 이전 값인 2가 남아 있을 수도 있기 때문입니다. 재실행 직후 앱에 3이 유지되는지와 통신 재개 후 서버가 최종적으로 3이 되는지를 나눠 판정하도록 했습니다.

03-5 앱 강제 종료 후 단수 기록 복구 (TC-CNT02-01)

후속 조치와 재검증

이 검증에서 별도 결함 수정은 필요하지 않았습니다. 이후에도 단수 변경과 저장을 반복 확인하도록, Appium의 단수 증가와 감소 테스트 및 pytest 기반 API의 단수 저장 테스트에 관련 확인 항목을 연결했습니다.

이 자동화가 Charles로 응답 대기 상태를 만든 뒤 앱을 강제 종료하는 절차 전체를 대신하는 것은 아닙니다. 응답 중단 시점에 따른 복구 확인은 수동 검증 조건으로 구분했습니다.

이 사례에서 배운 점

앱에 값이 보이는 시점과 서버 저장이 끝난 시점을 구분해야 합니다. 통신 중간 상태만으로 기록 유실을 판정하지 않고, 앱 재실행 후 보존과 이후 동기화 결과를 함께 확인했습니다.

근거: TC-CNT02-01 설계와 실행 판정 / 실행 기록: 강제 종료 후 화면과 서버 및 로컬 값 확인

04 결과와 회고: 결함을 반복 검증 기준으로 연결

기대 결과 → 실행 판정 → 결함 → 수정 → 회귀 항목으로 추적했습니다.

수동 검증 결과와 후속 관리

항목	결과와 관리 방식
수동 실행	테스트 케이스 21건의 실행 결과 기록
수정 후 반복 확인	도안 교체 시 드로잉 보존과 삭제는 Appium으로, 오류 항목이 섞인 목록의 정상 조회는 pytest 기반 API 테스트로 확인
자동화 실패 분석	오류 로그와 실패 화면, 해당 테스트만 다시 실행한 결과를 비교해 테스트 코드 오류와 실행 환경 문제를 구분

프로젝트를 통해 배운 점

1. 테스트 설계에는 기대 결과를 정하는 과정도 포함된다. 도안 위치가 초기화되는 현상을 발견했을 때, 사전 조회 후 복귀와 홈 탭 이동 후 복귀의 요구사항 범위가 달랐습니다. 사용자가 작업을 이어가는 상황을 기준으로 탭 복귀 시의 기대 결과를 추가했습니다. 구현된 동작을 그대로 정답으로 삼기보다, 사용 목적과 요구사항을 먼저 확인해야 한다는 점을 배웠습니다.
2. DB의 정합성 확인을 앱의 사용 상태까지 확장해야 한다. 서버가 현재 계정의 데이터만 반환해도 앱에는 이전 계정의 기록이 남을 수 있었습니다. 단수 변경 후에는 화면 반영 시점과 서버 저장 완료 시점도 달랐습니다. DB 값과 API 응답뿐 아니라 화면 이동, 계정 변경, 앱 종료 전후의 상태를 함께 대조하는 관점으로 검증 범위를 넓혔습니다.

04 결과와 회고

프로젝트를 통해 배운 점 (계속)

3. 자동화 결과는 통과와 실패의 이유까지 확인해야 한다. 프로젝트 기록에는 앱의 날짜 선택 동작을 잘못 가정한 테스트 코드 오류와, 장시간 실행 중 자동화 연결 문제로 발생한 실패가 있었습니다. 반대로 통과한 테스트도 실제로 어떤 조건을 검사했는지 확인해야 했습니다. 실행 결과를 해석할 때 오류 로그, 실패 화면, 개별 재실행 결과와 검사 조건을 함께 보는 것이 중요했습니다.
4. AI가 작성한 테스트도 기대 결과를 검토해야 한다. 기존 기록에는 AI가 작성한 테스트가 서버 저장 거부 시 기록이 사라지는 동작을 정상으로 검사한 사례가 있습니다. 테스트가 통과해도 사용자 기록을 보존한다는 요구사항을 충족하지 못할 수 있다는 뜻입니다. AI가 만든 코드와 문서를 검토할 때는 실행 성공 여부뿐 아니라 기대 결과가 기획 의도에 맞는지도 대조해야 한다는 점을 배웠습니다.

아쉬웠던 점과 개선 방향

1. 성능 검증을 충분히 진행하지 못했다. 기능 동작과 데이터 보존을 우선해, 프로젝트 수나 도안 파일 크기에 따른 목록 조회 시간과 도안 로딩 시간은 체계적으로 측정하지 못했습니다. 후속 검증에서는 데이터 규모별 조건과 응답 시간 기준을 정하고, 실제 기기에서 로딩 시간과 메모리 사용량을 확인하겠습니다.
2. 모바일 환경의 검증 범위를 더 넓힐 필요가 있다. 실기기 수동 테스트를 수행했지만, 반복 자동화는 시뮬레이터 중심이었습니다. 다음 검증에서는 실제 기기와 OS 조합을 정하고, 앱 종료와 복귀, 통신 중단 후 복귀, 권한 거부 후 재시도를 우선 확인하겠습니다. 화면 크기와 접근성도 별도 검증 범위로 정리하겠습니다.

상세 결과: 회차별 실행과 재검증 원장 / 상세 설계: 자동화 구성과 결함 연결 / 회고 근거: 요구사항 보완과 테스트 검토 과정